Практическая работа 2: Подсчёт объектов на последовательности изображений с использованием алгоритма SORT

Дисциплина: Экстремальная робототехника
Уровень: Магистратура, 1 курс

Дедлайн: занятие 14.03.2026

Пропуск дедлайна: оценка умножается на 0.7

(крайний срок сдачи – 14.03.2026, дальше работы не принимаются)

1. Цель работы

Целью данной практической работы является освоение алгоритма SORT (Simple Online and Realtime Tracking) для задачи подсчёта статических объектов (букс) на последовательности изображений, полученных с бортовой камеры мобильного робота.

Студенты научатся:

1. Обрабатывать покадрово видеопоследовательность, представленную набором изображений
2. Использовать детектор формата YOLO
3. Применять алгоритм SORT для подсчета уникальных объектов
4. Оценивать влияние параметров трекера и детектора на точность подсчёта

2. Теоретические основы

2.1. Алгоритм SORT

SORT (Simple Online and Realtime Tracking) – это метод многообъектного слежения, основанный на комбинации детектора и фильтра Калмана. Он состоит из четырёх основных этапов:

1. **Детекция** – на каждом кадре с помощью предобученной нейронной сети (в нашем случае YOLO) получают ограничивающие рамки (bounding boxes) объектов заданного класса.
2. **Предсказание состояния** – для каждого активного трека с помощью линейной модели постоянной скорости предсказывается его положение на следующем кадре. Состояние трека описывается вектором:
   $x =$ ¿где (u, v) – координаты центра рамки, s – площадь рамки, r – соотношение сторон (считается постоянным), $\dot{u}, \dot{v}, \dot{s}$ – соответствующие скорости.

3. **Сопоставление детекций с треками** – с помощью венгерского алгоритма решается задача назначения, где стоимость назначения равна метрике пересечения (IoU) между предсказанной рамкой трека и детекцией. Детекции, не попавшие ни в один трек, инициируют новые треки; треки, не получившие назначения, через заданное число кадров удаляются.
4. **Обновление состояния** – для успешно сопоставленных пар выполняется коррекция фильтра Калмана по измерению.

2.2. Метрика IoU

Intersection over Union (IoU) используется как мера близости двух рамок:

$$\mathrm{IoU} = \frac{\text{площадь пересечения}}{\text{площадь объединения}}$$

В SORT стоимость назначения определяется как $1 - \mathrm{IoU}$. Чем выше IoU, тем меньше стоимость.

2.3. Управление треками

- **Создание трека:** если детекция не сопоставлена ни с одним существующим треком, создаётся новый трек с начальным состоянием, определённым по этой детекции. Трек переводится в состояние «предварительный» до тех пор, пока не наберёт достаточное количество подтверждений (параметр min_hits).
- **Удаление трека:** если трек не получает назначения в течение заданного числа кадров (параметр max_age), он удаляется.

2.4. Подсчёт уникальных объектов

Для подсчёта букс, можно использовать простой подход: каждый подтверждённый трек увеличивает счётчик уникальных объектов.

3. Описание данных

3.1. Папка с изображениями

Входные данные представляют собой набор изображений в формате .png, расположенных в одной папке. Имена файлов содержат порядковый номер кадра, что обеспечивает хронологическую последовательность. Пример: rgb_175.png – где 175 это номер кадра. В наборе данных присутствуют кадры с номерами от 175 до 863.

3.2. Предобученная модель YOLOv8n

Предоставлена предобученная модель YOLOv8n (mega_hodov2.pt), которая уже умеет детектировать объекты класса «**букса»**. Метка класса «букса» – 4.

4. Задание

Необходимо разработать систему подсчёта статических объектов (букс, класс 4) на последовательности изображений, используя предобученную YOLOv8n и алгоритм трекинга SORT (или его улучшенную версию).

Работа включает следующие этапы:

- **Подготовка данных и детекция:** загрузить и отсортировать в хронологическом порядке все изображения из заданной папки. С помощью YOLOv8n выполнить детекцию объектов класса 4 на каждом кадре. Визуализировать примеры детекций.
- **Реализация трекера:** реализовать алгоритм SORT, используя фильтр Калмана. Обеспечить инициализацию, обновление и удаление треков. Реализовать подсчёт уникальных объектов: каждый уникальный трек увеличивает счётчик.
- **Анализ результатов:** запустить трекер на всей последовательности, визуализировать каждый кадр с наложенными рамками и присвоенными ID треков. Вывести итоговое количество подсчитанных букс, сравнить с истинным числом.

5. Требования к отчёту

Можно оформить как отдельный вордовский документ, так и просто все это описать в **jupyter notebook (предпочтительный вариант).**

Отчёт должен содержать:

1. **Титульный лист:** название работы, ФИО, дата
2. **Введение:** краткое описание задачи и данных
3. **Практическая часть:**
   - **Подготовка данных** – анализ последовательности, примеры детекций.
   - **Реализация** – объяснение ключевых моментов.
   - **Результаты** – визуализация трекинга, итоговое количество объектов, графики активных треков.
4. **Выводы:**
   - Какие сложности возникли, как их преодолели.
   - Ограничения реализованного подхода и возможные улучшения.
5. **Приложения:** исходный код (Python)

6. Критерии оценивания

Критерий	Максимум баллов	Описание
- Корректная загрузка и сортировка изображений	1	Кадры идут в правильном порядке
- Выполнение детекции YOLO	1	Нужный класс, правильная визуализация
- Реализация трекера	4	

- Визуализация трекинга	1	Графики траекторий
- Выводы	1	
Итого	8	

Для получения 10 баллов необходимо применить продвинутый вариант трекера (на Ваш выбор) и представить сравнение с базовым SORT.

7. Полезные ресурсы

Библиотеки Python

- filterpy – готовые реализации Kalman, EKF, UKF
- ultralytics – YOLO
- numpy – матричные операции
- matplotlib – визуализация
- scipy – математические функции
- opencv – вывод изображения

Установка:

pip install filterpy pandas numpy matplotlib scipy ultralytics opencv-python

Рекомендуемые ссылки

- https://arxiv.org/abs/1602.00763 – SORT
- https://filterpy.readthedocs.io/en/latest/ – библиотека filterpy
- https://docs.ultralytics.com/ – YOLO